Phase 3 — Load

Created	@February 27, 2026 4:28 PM
Tags	

วัตถุประสงค์ของ Phase 3

Phase 3 มีวัตถุประสงค์หลัก 2 ประการ คือ

1. เพื่อโหลดข้อมูลที่ผ่านการ Transform แล้วเข้าสู่ตาราง `fraud_clean_knime` บน MySQL
2. เพื่อจับเวลาขั้นตอน Load แยกตามขนาด 10K, 100K และ 1M แถว และบันทึกผล

Phase นี้มี 6 Node (Node 9-14)ทำตามลำดับเลย

Node 9 : MySQL Connector

MySQL Connector Node ทำหน้าที่เป็น **ตัวสร้างการเชื่อมต่อ (Connection Object)** ระหว่าง KNIME กับ MySQL Database โดยเฉพาะ โดยตัว Node เองไม่ได้ดึงหรือเขียนข้อมูลใดๆ เพียงแค่ทำหน้าที่การเชื่อมต่อ เท่านั้น ซึ่ง MySQL Connector Node จะส่งการเชื่อมต่อไปให้ Node ถัดไปใช้งาน

ขั้นตอนที่ 1 : เพิ่ม Node

1. พิมพ์ `MySQL Connector` ใน Search Node
2. ลากมาวางใต้ Workflow (แยกออกมา ยังไม่ต้องต่อสาย)

ขั้นตอนที่ 2 : ตั้งค่า

คลิกขวากด configure แล้วตั้งค่า:

Field	ค่า
Hostname	`localhost`
Port	`3306`
Database name	`fraud_dl`
Username	ชื่อ user MySQL
Password	รหัสผ่าน

ขั้นตอนที่ 3 : Execute ก่อนยังไม่ต้องต่อสาย

คลิกขวา → **Execute** ถ้าสำเร็จ Node จะเป็นสีเขียว ✅

Node 10 : DB Writer

DB Writer Node ทำหน้าที่ **เขียนข้อมูลจาก KNIME ลงสู่ตารางใน Database** โดยรับข้อมูลที่ผ่านการ Transform มาแล้วจาก Node ก่อนหน้า และรับการเชื่อมต่อจาก MySQL Connector เพื่อรู้ว่าต้องเขียนลง Database ไหน

ขั้นตอนที่ 1 : เพิ่ม Node

1. พิมพ์ `DB Writer` ใน Search Node
2. ลากมาวางต่อจาก Node 8

ขั้นตอนที่ 2 : ต่อสาย

```
[Node 8: Java Snippet Timer Stop for transform phase] ──►
[Node 10: DB Writer]

|
[Node 9: MySQL Connector] ──────────────────────────────
──────────┘
```

ขั้นตอนที่ 3 : ตั้งค่า DB Writer

คลิกขวากด configure แล้วตั้งค่า:

Field	ค่า
Schema name	`fraud_dl`
Table name	`fraud_clean_knime`

- **If table exists:** เลือก Append rows
- **Columns to write:**

  ต้องย้าย column ที่ไม่มีใน MySQL ออกไปฝั่ง Excludes:

  1. `column1`
  2. `street`

3. `zip`
4. `lat`
5. `long_col`
6. `unix_time`
7. `merch_lat`
8. `merch_long`
9. `extract_start_time`
10. `extract_sec`
11. `transform_sec`

เลื่อนดู Includes ทั้งหมดแล้วเทียบกับ Schema MySQL ที่สร้างไว้:

```
trans_datetime, cc_num_hash, merchant, category, amt,
first_hash, last_hash, gender, city, state,
city_pop, job, dob_hash, trans_num, is_fraud
```

> column ที่เหลือใน Includes จะถูก Map เข้า MySQL อัตโนมัติ แต่ชื่อต้องตรงกัน ถ้าไม่ตรงจะ Error

- **ส่วน Output:** เอาติ๊ก ✅ ออก
  - ติ๊ก ✅ **Append status column** ออก
  - ติ๊ก ✅ **Append error column** ออก

กด **OK**

ขั้นตอนที่ 4 : Execute

คลิกขวา → **Execute** ถ้าสำเร็จ Node จะเป็นสีเขียว ✅

Node 11 : Java Snippet (Load Timer Stop)

Node นี้ทำหน้าที่ **หยุดจับเวลาของ Phase Load** และคำนวณเวลาที่ใช้ในแต่ละ Phase ทั้งหมด เพื่อเตรียมบันทึกลง CSV

ขั้นตอนที่ 1 : เพิ่ม Node

1. พิมพ์ `Java Snippet` ใน Search Node

2. ลากมาวางต่อจาก **Node 10 (DB Writer)**

ขั้นตอนที่ 2 : ต่อสาย

```
[Node 10: DB Writer] ──► [Node 11 Java Snippet: Load Timer Stop]
```

ขั้นตอนที่ 3 : ตั้งค่า

ส่วนที่ 1 : เพิ่ม Input Column

กด Add ในส่วน Input 2 ครั้ง:

Name	Java Type	Java Field
`extract_start_time`	LocalDateTime	`in_extract_start_time`
`transform_sec`	Double	`in_transform_sec`

ส่วนที่ 2 : เพิ่ม Output Columns

กด Add ในส่วน Output 1 ครั้ง:

Append	Output Type	Java Field
`load_sec`	Number (Float)	`out_load_sec`
`dataset`	String	`out_dataset`

ขั้นตอนที่ 4 : ใส่ Code

```
long load_end = System.currentTimeMillis();

// แปลง extract_start_time (LocalDateTime) เป็น millisecond
long extract_start_ms = in_extract_start_time
    .atZone(java.time.ZoneId.systemDefault())
    .toInstant()
    .toEpochMilli();

// คำนวณ load_sec
double total_elapsed = (load_end - extract_start_ms) / 1000.0;
double load_sec = total_elapsed - in_transform_sec;
```

```
out_load_sec = load_sec;
out_dataset = "10k";
```

ขั้นตอนที่ 5 : Execute

กด **Reset all** → **Execute all**

คลิกขวา → **Execute** แล้วเช็คว่ามี column `load_sec` มีค่าเป็นวินาที และ column `dataset` ปรากฏขึ้นมา

Node 12 : Table Row to Variable

Node นี้ทำหน้าที่แปลงข้อมูลในแถวแรกของ Table ให้กลายเป็น Flow Variable (ตัวแปรที่อยู่นอก Table) สรุปง่ายๆคือ ทำให้ข้อมูลแถวแรกในตารางกลายเป็นตัวแปรที่อยู่นอกตารางเพื่อให้ Node ที่อยู่ ถัดไปในทุกจุดของ **Workflow** สามารถดึงตัวแปรนั้นไปใช้ได้เลย ในโปรเจคนี้แปลงค่า extract_sec, transform_sec, load_sec และ dataset จาก Row แรกเป็น Flow Variable

ขั้นตอนที่ 1 : เพิ่ม Node

1. พิมพ์ `Table Row to Variable` ใน Search Node
2. ลากมาวางต่อจาก Load Timer Stop

ขั้นตอนที่ 2 : ต่อสาย

```
[Node 11 Java Snippet: Load Timer Stop] ── [Node 12 : Table
Row to Variable Node]
```

ขั้นตอนที่ 3 : ตั้งค่า

1. ย้ายไปฝั่ง Excludes ให้เหลือแค่ 4 ตัวใน Includes
2. กดปุ่ม **<<** เพื่อย้ายทุกอย่างไป Excludes ก่อน แล้วค่อยย้ายกลับมาแค่ 4 ตัวนี้:

   ```
   extract_sec
   transform_sec
   ```

```
load_sec
dataset
```

3. กด **Apply** หรือ **Apply and Execute**

Node 13 : Variable to Table Row

Node นี้ทำหน้าที่ **ตรงข้ามกับ Table Row to Variable** เลยคือแปลง Flow Variable ที่ลอยอยู่นอก Table ให้กลับมาเป็น **แถวข้อมูลใน Table** แทนในโปรเจคนี้ Node นี้จะรับค่า extract_sec, transform_sec, load_sec และ dataset จาก Node 12 : Table Row to Variable กลับมาเป็นแถวข้อมูลในตารางอีกครั้ง ทั้งหมดนี้เพื่อให้ CSV Writer ที่รับได้แค่ Table ธรรมดาสามารถบันทึกค่า Timing ทั้ง 3 Phase ลงไฟล์ได้

ขั้นตอนที่ 1 : เพิ่ม Node

1. พิมพ์ `Variable to Table Row` ใน Search Node
2. ลากมาวางต่อจาก **Table Row to Variable**

ขั้นตอนที่ 2 : ต่อสาย

```
[Node 12 : Table Row to Variable Node] ── [Node 13 : Variab
le to Table Row]
```

ขั้นตอนที่ 3 : ตั้งค่า

ย้าย 3 ตัวนี้จาก Excludes → Includes:

- `extract_sec`
- `transform_sec`
- `load_sec`
- `dataset`

โดยคลิกเลือกทีละตัวแล้วกดปุ่ม `>` จากนั้นกด **Apply** หรือ **Apply and Execute**

Node 14 : CSV Writer

CSV Writer Node ทำหน้าที่ **เขียนข้อมูลจาก Table ออกมาเป็นไฟล์ .csv** บนเครื่องของเรา

ขั้นตอนที่ 1 : เพิ่ม Node

1. พิมพ์ `CSV Writer` ใน Search Node
2. ลากมาวางต่อจาก **Variable to Table Row**

ขั้นตอนที่ 2 : ต่อสาย

```
[Node 13 : Variable to Table Row] ──► [Node 14 :  CSV Write
r]
```

ขั้นตอนที่ 3 : ตั้งค่า

1. เลือก location ที่อยากให้ไฟล์ csv แสดง
2. ใส่ชื่อไฟล์ในช่อง **Name**:

   ```
   knime_10k_timing_log.csv
   ```

   กด Choose file
3. **If exists :** Overwrite
4. กด **Apply** หรือ **Apply and Execute**

ตรวจสอบผลใน MySQL Workbench:

รันโค้ดนี้:

```
USE fraud_dl;

-- ตรวจจำนวนแถว (ต้องได้ 10000)
SELECT COUNT(*) FROM fraud_clean_knime;

-- ตรวจ hash ความยาว 64 ตัวอักษร
SELECT
        trans_datetime,
    cc_num_hash,
    first_hash,
    last_hash,
    dob_hash
```

```
FROM fraud_clean_knime LIMIT 3;

-- ตรวจไม่มี amt ติดลบ
SELECT COUNT(*) FROM fraud_clean_knime WHERE amt <= 0;
```

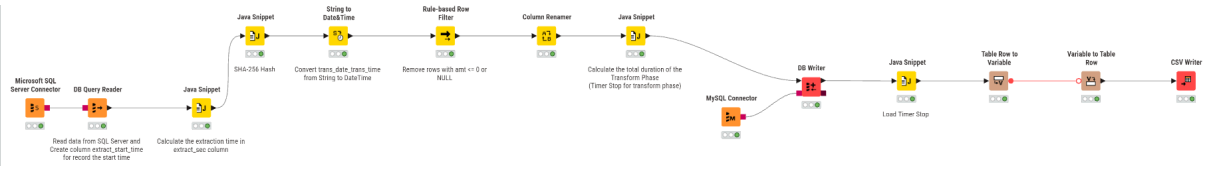
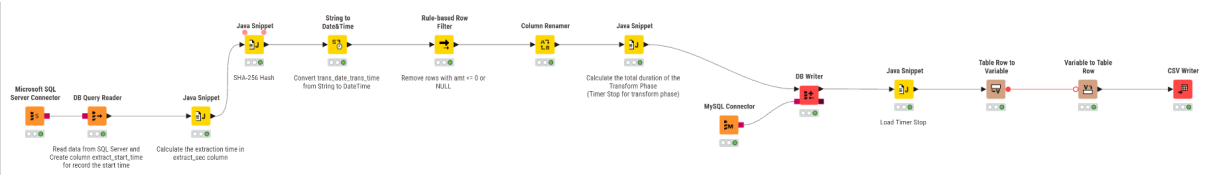
และเปิด timing_log_knime.csv ในโฟลเดอร์ที่กำหนด location ในโปรเจคนี้กำหนดไว้ที่โฟลเดอร์ data

---

ก่อนรัน 100K หรือ 1M:

1. ใน MySQL รัน TRUNCATE TABLE ก่อน โดยรันโค้ดนี้:

   ```
   USE fraud_dl;
   TRUNCATE TABLE fraud_clean_knime;
   ```

2. เปลี่ยน Node 2 , Node 11 และ Node 14 เป็น "100k" หรือ "1m"
3. กด **Reset all** → **Execute all**

ภาพประกอบการทดสอบ ETL Pipeline ด้วย KNIME